VoiceOps 2026: Capstone Technical Report

Modular Lexical RAG Support Agent | Author: Balasundaram Shanmugaraj | IIT Guwahati × Gnani.ai
Repository: https://github.com/balasundaramshanmugaraj/VoiceOps_Project

1. Executive Summary & Architecture

This capstone project implements a production-grade, modular customer support agent for the synthetic Indian e-commerce platform **ZentroMart**. Designed with strict separation of concerns, the architecture decouples lexical document retrieval, policy hierarchy reasoning, multi-turn conversation memory, and voice synthesis/transcription. The core retrieval layer relies on Okapi BM25 with section-level chunking that preserves section IDs (e.g., [RET-2], [DWM-3]), enabling end-to-end evidence tracking and auditability. The conversational core enforces two-tier identity verification before disclosing order records, enforces holiday-aware business day deadlines anchored to **20 October 2026**, and routes high-value damage claims directly to human Priority Resolution teams via ticket creation.

Component	Implementation Mechanism	Key Design Distinction
Document Indexing	Section-aware Markdown frontmatter & heading parser	Preserves [SEC-ID], metadata status, and effective dates
Retrieval Engine	BM25Okapi lexical index with query tokenization	Fast, deterministic, zero-hallucination lexical recall
Precedence Gate	Document-rank sorting according to KB-00.1 hierarchy	Policy > Agent Guide > Help > FAQ > Marketing
Conversation State	Stateful session manager with verification gate	Enforces max 1 verification retry; hides private data
Voice Pipeline	Gnani STT / TTS REST integration (timbre-v2.5, Kaveri)	10-line voice turn, ear-friendly normalization, batch run

2. Implementation of the Four Capstone Steps

Step 1 — Prompt-Only LLM Baseline: Implemented in src/baseline.py. The agent adopts a generic ZentroMart support persona that introduces itself as an AI assistant. It answers the 52 official practice questions without document retrieval. Lacking internal policy documents, it guesses standard market policies (e.g., claiming 5-7 days for wallet refunds or 7 days for notebooks), providing an empirical baseline.

Step 2 — Retrieval-Augmented Generation (RAG): Implemented in src/rag_agent.py and src/retriever.py. The 21 Markdown knowledge documents in kb/ are parsed into discrete sections tagged with section IDs and precedence ranks. Lexical BM25 retrieval finds relevant sections and extracts catalog rows from products.csv for SKUs. Policies with status current supersede outdated policies, avoiding traps like FAQ-1. Unanswerable questions trigger graceful abstentions offering human colleagues.

Step 3 — Conversational Agent and Tools: Implemented in src/conversation_agent.py and src/tools_adapter.py. Manages multi-turn state across customer turns. When an order ID is mentioned, it performs an unverified lookup and asks for the customer's phone suffix (last 4 digits) or delivery pincode. Upon successful verification, it unlocks order details; upon failure, it allows at most 1 retry before locking. For high-value damaged items (>■10,000), it executes create_ticket() and confirms handover only upon created=True.

Step 4 — Voice with Gnani.ai: Implemented in src/voice_adapter.py. Integrates Gnani's official gnani-vachana SDK using GnaniSTTClient and GnaniTTSClient. Audio inputs are transcribed, processed through the Step 3 agent, and synthesized into natural speech audio using voice **Kaveri** and model **timbre-v2.5**. Handles batch audio processing via python run.py --input-dir ... --output-dir ..., saving transcripts, responses, audio files, evidence, and tool execution logs.

3. Empirical Evaluation Results

All 52 official practice questions and 23 multi-turn conversations (96 customer turns) were executed. The table below compares the actual measured performance of Step 1 Baseline against Step 2 Lexical RAG and Step 3 Conversational handling.

Evaluation Metric	Step 1: Baseline	Step 2: Lexical RAG	Step 3: Multi-Turn	Improvement / Note
Decision Accuracy	88.5%	88.5%	97.9%	+9.4% multi-turn precision
Fact Coverage Rate	1.9%	51.9%	96.9%	+95.0% grounded facts

Gold Evidence Recall	0.0%	65.4%	91.2%	Section ID provenance verified
Abstention on Unsupported	0.0%	60.0%	100.0%	+60.0% safe decline of traps
Tool-Call Accuracy	N/A	N/A	100.0%	Strict IDV gating & escalation
Average Turn Latency	12 ms	28 ms	35 ms	Real-time capable performance

4. Technical Analysis: What Worked & What Failed

What Worked Exceptionally Well:

1. **Strict Policy Precedence (KB-00.1):** Grounding answers in policy documents rather than FAQ summaries eliminated false promises (e.g., Question PQ0003: FAQ-1 erroneously states wallet refunds take 5-7 days; our precedence gate selected REF-2, delivering the correct 2-hour instant refund timeline).
2. **Grandfathered Policy Handling:** In PQ0019, the agent correctly identified that electronics delivered prior to 1 August 2026 fall under superseded version 3 (10-day window) rather than version 4 (7-day window).
3. **Multi-turn Verification Security:** In conversations PC002 and PC003, private customer details were never leaked before verification. When Kabir provided '4533', verification passed seamlessly, and the TV damage claim was escalated via ticket.

Failure Modes & Error Categorization:

1. **Vocabulary Mismatch in Pure Lexical Search:** Lexical BM25 struggles when customers use colloquial synonyms not found in policy text (e.g. 'send it back' vs 'reverse pickup'). In such queries, BM25 scored low, lowering raw retrieval recall to 65.4%.
2. **Complex Date Arithmetic in Hindi:** Hinglish queries like 'credit card wala refund 17 October ko shuru hua' required both parsing Hindi numerals and executing holiday skips (skipping 19 October warehouse festival). While our rule engine handled this, purely neural LLMs without tools hallucinated calendar dates.

5. Voice Integration with Gnani.ai & Batch Audio Command

The voice layer implements Gnani.ai's official Python SDK. For each audio turn, GnaniSTTClient transcribes incoming speech. If speech is unclear or silent, the agent recovers gracefully by uttering *'Sorry, I didn't catch that. Could you say it again?'* Valid transcripts are routed through the Step 3 agent, and the response is synthesized via GnaniTTSClient using voice **Kaveri**. The batch command python run.py --input-dir ... --output-dir ... was tested against test fixtures, achieving 100% processing success, producing reply WAV files, and logging execution status in batch_processing_summary.json.

6. Limitations & Future Roadmap

• **Hybrid Dense Retrieval:** Integrating semantic dense vector embeddings alongside BM25 lexical search will bridge colloquial lexical gaps.
• **Live Multi-accent Speech Tuning:** Real-world telephony introduces acoustic noise and varied Indian English/Hindi accents; future tuning will leverage Gnani's streaming WebSocket client with noise suppression.
• **Proactive Handover Tracking:** Extending ticket statuses to notify customers via SMS or WhatsApp upon resolution.